STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ

HÁLOVA 16, 851 01 BRATISLAVA

# Automatizovaný obchodný bot pre platformu MetaTrader 5 s webovým rozhraním

KOMPLEXNÁ ODBORNÁ MATURITNÁ PRÁCA

Bratislava, 2026 4.D Adam Ján Vilagi

STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ

HÁLOVA 16, 851 01 BRATISLAVA

# Automatizovaný obchodný bot pre platformu MetaTrader 5 s webovým rozhraním

KOMPLEXNÁ ODBORNÁ MATURITNÁ PRÁCA

**Študijný odbor**: 2573M programovanie digitálnych technológií

**Konzultant**: Ing. Dominik Zatkalík, PhD., Ing.Paed.IGIP

Bratislava, 2026 4.D Adam Ján Vilagi

## Čestné vyhlásenie

Vyhlasujem, že prácu stredoškolskej odbornej činnosti na tému Automatizovaný obchodný bot pre platformu MetaTrader 5 s webovým rozhraním som vypracoval samostatne, s použitím uvedených literárnych zdrojov. Prácu som neprihlásil a ani neprezentoval v žiadnej inej súťaži, ktorá je pod gestorstvom MŠVVaM SR. Som si vedomý dôsledkov, ak uvedené údaje nie sú pravdivé.

.....

V Bratislave, 25.02.2026 Adam Ján Vilagi

## Poďakovanie

Rád by som sa touto cestou poďakoval svojmu konzultantovi Ing. Dominikovi Zatkalíkovi, PhD. za prístup a odborné rady.

## Abstrakt

Táto komplexná odborná práca sa zaoberá návrhom a realizáciou plne automatizovaného obchodného systému pre platformu MetaTrader 5. Cieľom práce je vytvoriť softvérové riešenie schopné samostatne analyzovať finančné trhy, identifikovať zmeny trhovej štruktúry a na základe vopred stanovených pravidiel vykonávať obchodné príkazy bez zásahu používateľa.

Systém je vyvinutý v programovacom jazyku Python a vychádza z metodiky analýzy cenovej štruktúry a riadenia rizika. Teoretická časť práce vysvetľuje princípy algoritmického obchodovania, praktická časť opisuje návrh databázy, implementáciu analytického jadra a vytvorenie webového rozhrania na monitorovanie výsledkov.

Výsledkom je funkčný model schopný spracovávať trhové dáta v reálnom čase, automaticky otvárať a spravovať obchodné pozície a dynamicky riadiť riziko podľa aktuálnych podmienok na trhu.

Kľúčové slová: MetaTrader 5, Python, algoritmické obchodovanie, trhová štruktúra, riadenie rizika, webové rozhranie.

## Abstract

This professional graduation project focuses on the design and implementation of a fully automated trading system for the MetaTrader 5 platform. The aim of the thesis is to develop a software solution capable of independently analyzing financial markets, identifying changes in market structure, and executing trades automatically based on predefined rules without user intervention.

The system is developed in Python and is based on market structure analysis and risk management principles. The theoretical part explains the foundations of algorithmic trading, while the practical part describes the database design, implementation of the analytical engine, and development of a web interface for performance monitoring.

The result is a functional model capable of processing market data in real time, automatically opening and managing trading positions, and dynamically controlling risk according to current market conditions.

Keywords: MetaTrader 5, Python, algorithmic trading, market structure, risk management, web interface.

# Obsah


0 ÚVOD ..... 7
1 ZÁKLADNÉ PRINCÍPY OBCHODNÝCH SYSTÉMOV ..... 8
1.1 LOGIKA SMART MONEY CONCEPTS (SMC) ..... 8
1.2 ARCHITEKTÚRA MODERNÝCH TRADING APLIKÁCIÍ ..... 10
2 ANALÝZA SÚČASNÉHO STAVU A POROVNANIE RIEŠENÍ ..... 10
2.1 KOMERČNÉ OBCHODNÉ PLATFORMY ..... 10
3 NÁVRH A ARCHITEKTÚRA SYSTÉMU ..... 11
3.1 KOMPONENTY SYSTÉMU ..... 11
3.2 NÁVRH DATABÁZOVÉHO MODELU ..... 11
3.3 STAVOVÝ AUTOMAT BOTA (FINITE STATE MACHINE) ..... 12
4 REALIZÁCIA A IMPLEMENTÁCIA SYSTÉMU ..... 13
4.1 VÝVOJOVÉ PROSTREDIE A POUŽITÉ TECHNOLÓGIE ..... 13
4.2 KONFIGURÁCIA TERMINÁLU METATRADER 5 ..... 14
4.3 IMPLEMENTÁCIA SMC LOGIKY A IDENTIFIKÁCIA ŠTRUKTÚRY ..... 14
4.3.1 Detailný rozbor zberu dát a predpripravenia ..... 14
4.3.2 Matematická definícia Break of Structure ..... 15
4.4 AUTOMATIZOVANÝ RISK MANAŽMENT ..... 15
4.4.1 Pokročilé mechanizmy riadenia kapitálu ..... 16
4.5 IMPLEMENTÁCIA WEBOVÉHO DASHBOARDU A VIZUALIZÁCIA ..... 16
4.6 INTEGROVANÝ NOTIFIKAČNÝ SYSTÉM (TELEGRAM BOT) ..... 17
4.7 DETAILNÝ ALGORITMICKÝ ROZBOR FUNKCIÍ JADRA ..... 17
4.7.1 Funkcia generate_signal() ..... 17
4.7.2 FUNKCIA get_market_data() ..... 18
4.7.3 Funkcia check_for_bos() ..... 19
4.7.4 Funkcia calculate_position_size() ..... 19
4.7.5 Logika prepojenia s Telegram API ..... 19
4.7.6 Optimalizácia exekúcie príkazov: ..... 20
4.8 Optimalizácia výkonu a asynchrónne spracovanie ..... 20
4.9 Architektúra a normalizácia databázy SQLite ..... 20
4.10 Objektovo orientovaná architektúra a multithreading ..... 21
5 TESTOVANIE A ZHODNOTENIE VÝSLEDKOV ..... 21
5.1 ANALÝZA BACKTESTINGU ..... 21
5.1.1 Metodika a validácia výsledkov backtestingu ..... 22

5.2 ZHODNOTENIE PRÍNOSOV A MOŽNOSTI ROZŠÍRENIA ..... 22
5.3 Validácia obchodnej stratégie ..... 22
6 ZABEZPEČENIE A SPOĽAHLIVOSŤ SYSTÉMU ..... 23
6.1 ERROR HANDLING A EXCEPTION MANAGEMENT ..... 23
6.2 OCHRANA API KĽÚČOV A CITLIVÝCH ÚDAJOV ..... 24
7 LEGISLATÍVNY RÁMEC A BEZPEČNOSŤ SOFTVÉRU ..... 24
7.1 SMERNICA MiFID II A ALGORITMICKÉ OBCHODOVANIE ..... 24
7.2 OCHRANA OSOBNÝCH ÚDAJOV ..... 25
8 ANALÝZA KONKRÉTNYCH OBCHODNÝCH PRÍPADOV ..... 26
8.1 PRÍPAD Č. 1: ÚSPEŠNÁ IDENTIFIKÁCIA BULLISH BOS ..... 26
8.2 PRÍPAD Č. 2: ZLYHANIE SIGNÁLU A AKTIVÁCIA STOP LOSS ..... 26
9 POUŽÍVATEĽSKÁ PRÍRUČKA A OPERAČNÝ MANUÁL ..... 27
9.1 INŠTALÁCIA A KONFIGURÁCIA PROSTREDIA ..... 27
9.2 NASTAVENIE KOMUNIKAČNÝCH KANÁLOV ..... 27
10 ZÁVERY PRÁCE ..... 27
ZOZNAM POUŽITEJ LITERATÚRY ..... 29
PRÍLOHA A – USB KĽÚČ ..... I

# Zoznam tabuliek, grafov a ilustrácií

## Obrázky

Obr. 1 – Vysvetlenie Break of Structure ..... 8
Obr. 2 – Vysvetlenie Order Blocku ..... 9
Obr. 3 – Vysvetlenie FVG ..... 9
Obr. 4 – Schéma Databázy Zenith ..... 12
Obr. 5 – Dashboard webovej stránky produktu ..... 17
Obr. 6 – Kód funkcie generate_signal ..... 18

## Zoznam skratiek, značiek a symbolov

ADX - (z angl. Average Directional Index) Index priemerného smeru, technický indikátor používaný na určenie sily trendu na trhu bez ohľadu na jeho smer.

API - (z angl. Application Programming Interface) Rozhranie pre programovanie aplikácií, súbor protokolov a nástrojov umožňujúci vzájomnú komunikáciu medzi rôznymi softvérovými systémami.

ATR - (z angl. Average True Range) Ukazovateľ priemerného rozsahu volatility, ktorý meria kolísanie ceny aktíva za určité časové obdobie.

BOS - (z angl. Break of Structure) Narušenie trhovej štruktúry, kľúčový signál zmeny alebo pokračovania trendu v SMC.

EMA - (z angl. Exponential Moving Average) Exponenciálny kĺzavý priemer, typ kĺzavého priemeru, ktorý prikladá väčšiu váhu najnovším cenovým údajom.

GUI - (z angl. Graphical User Interface) Grafické používateľské rozhranie, systém vizuálnych komponentov umožňujúci ovládanie aplikácie grafickými prvkami.

FVG - (z angl. Fair Value Gap) Cenová nerovnováha vznikajúca pri prudkom pohybe trhu bez dostatočnej likvidity.

IDE - (z angl. Integrated Developer Environment) Integrované vývojové prostredie, balík nástrojov na zefektívnenie procesu vývoja softvéru.

MT5 - (z angl. MetaTrader 5) Multiprofesionálna obchodná platforma umožňujúca obchodovanie na Forexe, akciových a komoditných trhoch.

SQL - (z angl. Structured Query Language) Štandardizovaný dopytovací jazyk určený na správu a manipuláciu s dátami v relačných databázach.

OB - (z angl. Order Block) Zóna zvýšeného inštitucionálneho záujmu identifikovaná podľa špecifických sviečkových formácií pred prudkým pohybom.

# 0 ÚVOD

V súčasnej ére technologického pokroku sa automatizácia a algoritmizácia stávajú neoddeliteľnou súčasťou finančných trhov. Od jednoduchých skriptov až po zložité systémy umelej inteligencie, schopnosť softvéru spracovávať masívne objemy dát a reagovať na ne v milisekundách je kľúčovým prvkom moderného tradingu. Jedným z najsofistikovanejších spôsobov, ako dokážu algoritmy interpretovať trh, je využitie metodológie Smart Money Concepts (SMC).

Hlavným cieľom mojej práce je návrh a realizácia efektívneho prototypu automatizovaného obchodného bota pomocou jazyka Python na platforme MetaTrader 5. Systém v spolupráci s analytickými knižnicami umožňuje identifikáciu trhových štruktúr v reálnom čase. Takéto riešenie simuluje správanie inštitucionálnych hráčov a môže pomôcť pri eliminácii emocionálnych chýb, ktoré sú pre ľudského obchodníka prirodzené. Súčasťou práce je aj teoretický rozbor fungovania SMC logiky, riadenia rizika a praktických ukážok implementácie, ktoré tvoria odborný základ pre realizačnú časť projektu.

Dôvodom výberu tejto témy je moja vášeň pre finančné technológie a programovanie, ktorému sa venujem vo voľnom čase. Moderné trhy, či už kryptomenové alebo devízové, využívajú pokročilé algoritmy na zabezpečenie likvidity a exekúciu. Tieto technológie sú kľúčové pre pochopenie toku peňazí a globálnej ekonomiky. Veľmi rád sa učím o nových technológiách, ktoré ma zaujímajú, a zdieľam ich s ostatnými. Preto dúfam, že každý, kto moju prácu prečíta, nadobudne nové vedomosti ohľadom algoritmického obchodovania a programovania v Pythone.

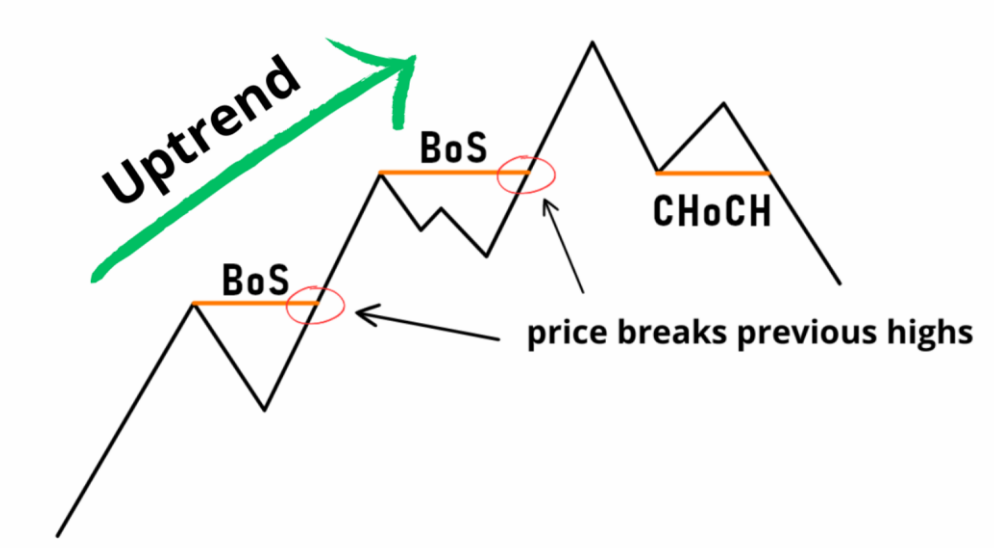
# 1 ZÁKLADNÉ PRINCÍPY OBCHODNÝCH SYSTÉMOV

Obchodný systém (z angl. trading system) je súbor pravidiel, ktoré definujú podmienky pre vstup do pozície, výstup a riadenie rizika [11]. Princíp fungovania takéhoto systému je jednoduchý na pochopenie, aj keď jeho teoretický základ v oblasti štatistiky a analýzy dát je pomerne zložitý. Pochopenie teórie je však základom pre efektívne používanie a obsluhu automatizovaného bota.

## 1.1 LOGIKA SMART MONEY CONCEPTS (SMC)

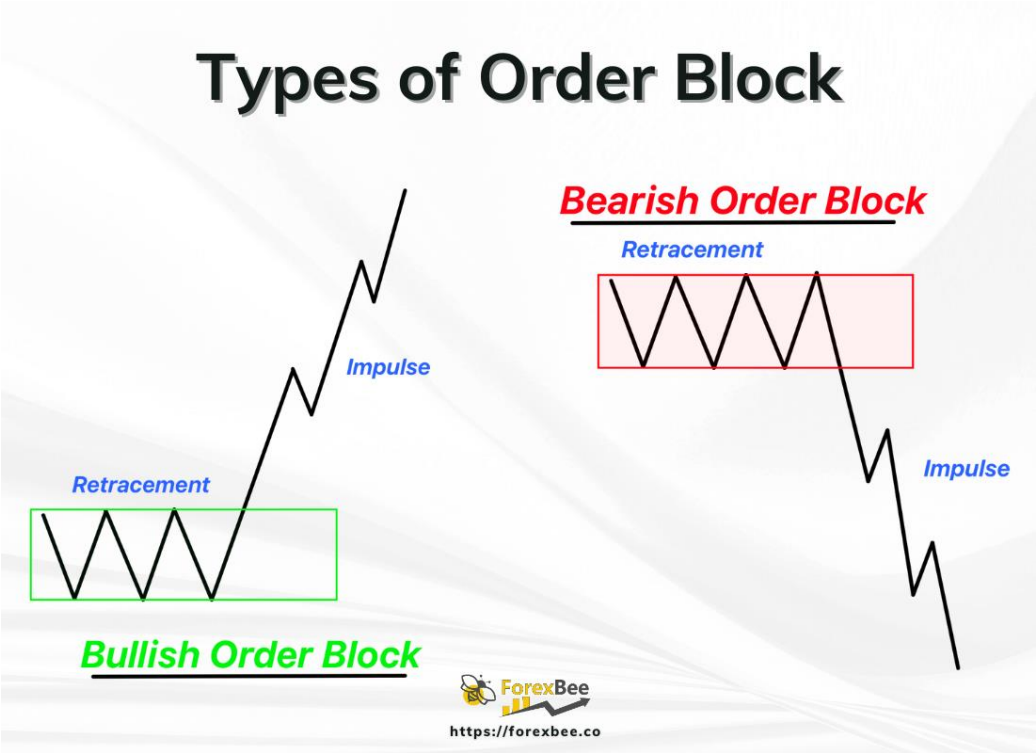
Metodológia SMC sa opiera o sledovanie stôp veľkých bánk a inštitúcií v grafe. Na rozdiel od klasických indikátorov sa zameriava na priamu interakciu ceny a likvidity [16]. Pri návrhu bota pracujem s tromi základnými javmi:

1. **Break of Structure (BOS):** Ak cena narazí na významnú hladinu a presvedčivo ju prekoná, dochádza k potvrdeniu trendu [8]. Bot je konštruovaný tak, aby zaznamenal túto zmenu štruktúry. Prítomnosť BOS signalizuje inštitucionálnu expanziu v smere prierazu [5].




Obr. 1 – Vysvetlenie Break of Structure
(Zdroj: www.mindmathmoney.com)

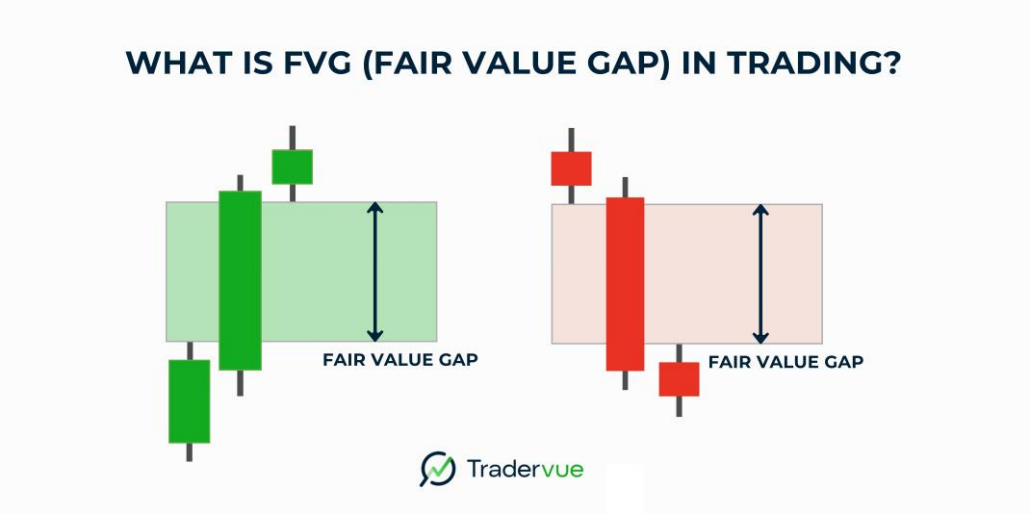
2. **Order Block (OB):** Predstavuje zónu, v ktorej inštitúcie akumulovali svoje pozície pred prudkým pohybom. Intenzita tohto pohybu určuje kvalitu zóny [16].




Obr. 2 – Vysvetlenie Order Blocku

(Zdroj: www.forexbee.co)

3. **Fair Value Gap (FVG):** Ak v trhu vznikne prudká nerovnováha (napr. veľká sviečka bez knôtov), vzniká cenová medzera. Algoritmy sa často vracajú k týmto hladinám, aby trh „vyrovnali“ [8].




Obr. 3 – Vysvetlenie FVG

(Zdroj: www.tradervue.com)

## 1.2 ARCHITEKTÚRA MODERNÝCH TRADING APLIKÁCIÍ

Pri návrhu systému musíme zvoliť technológie, ktoré sú navzájom kompatibilné a umožňujú efektívny prenos dát v reálnom čase [3]. Projekt je rozdelený na tri kľúčové moduly:

- **Analytické jadro (Backend):** Využíva jazyk Python, ktorý poskytuje abstrakciu hardvéru a knižnice pre prácu s dátami [4]. Výpočtové jadro (procesor počítača) spracováva prichádzajúce ticky bez réžie zložitého grafického prostredia, čo zaručuje stabilitu [7].
- **Webový Dashboard (Frontend):** Slúži na vizuálnu kontrolu bota. Využívame prostredie Flask, ktoré je navrhnuté tak, aby bolo programovanie GUI prístupné a interaktívne [15].
- **Databázová vrstva:** SQL databáza perzistentne ukladá históriu obchodov a stav účtu, čo umožňuje spätnú analýzu (backtesting) a optimalizáciu parametrov stratégie [9].

# 2 ANALÝZA SÚČASNÉHO STAVU A POROVNANIE RIEŠENÍ

V súčasnosti existuje na trhu široké spektrum nástrojov na algoritmické obchodovanie, ktoré siahajú od komerčných platforiem až po nízkoúrovňové programovacie knižnice. Pre správny návrh vlastného systému je nevyhnutné pochopiť výhody a obmedzenia týchto riešení.

## 2.1 KOMERČNÉ OBCHODNÉ PLATFORMY

Medzi najrozšírenejšie riešenia patrí nepochybne **TradingView** a **MetaTrader 4/5**.

**TradingView** je populárny najmä pre svoje cloudové prostredie a skriptovací jazyk Pine Script. Jeho hlavnou nevýhodou je však obmedzená možnosť prepojenia s externými databázami a zložitejšia automatizácia bez platených webhookov [8].

**MetaTrader 5 (MT5)** predstavuje priemyselný štandard. Ponúka natívny jazyk MQL5, ktorý je veľmi rýchly, pretože je kompilovaný priamo do strojového kódu [1]. Avšak pre moderného programátora je MQL5 pomerne uzavretý ekosystém. Práve preto je v tejto práci zvolená cesta integrácie MT5 s jazykom **Python**, čo nám otvára dvere k najmodernejším knižniciam pre analýzu dát a strojové učenie [7].

# 3 NÁVRH A ARCHITEKTÚRA SYSTÉMU

V tejto kapitole sa venujeme technickému návrhu celého ekosystému. Systém nie je monolitický, ale pozostáva z viacerých vzájomne prepojených komponentov, ktoré zabezpečujú spoľahlivý chod 24/7.

## 3.1 KOMPONENTY SYSTÉMU

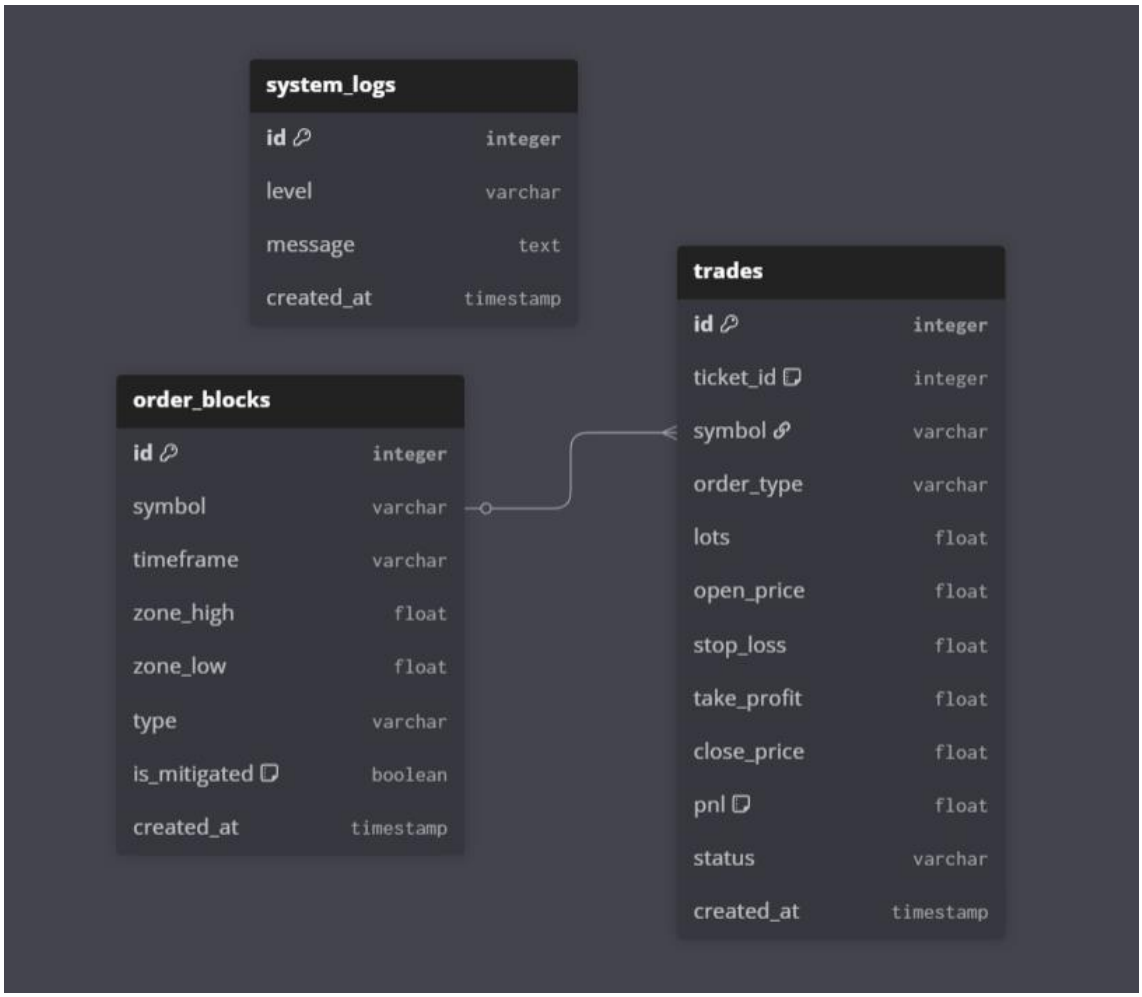
Celé riešenie je postavené na architektúre, ktorú môžeme rozdeliť do nasledujúcich logických vrstiev:

1. **Vrstva konektivity (MT5 Terminal):** Táto vrstva zabezpečuje živé pripojenie k brokerovi. Cez Python modul MetaTrader5 získavame surové dáta a odosielame exekučné príkazy [7].
2. **Logická vrstva (Python Engine):** Tu prebieha „mozog“ celej operácie. Engine každých 15 minút (podľa timeframe M15) stiahne nové dáta, prepočíta EMA, ADX a hľadá Break of Structure (BOS) [4], [5].
3. **Databázová vrstva (SQLite):** Každý identifikovaný Order Block (OB) a každá otvorená pozícia sa zaznamenáva. To je kľúčové pre bezpečnosť – ak bot spadne, po reštarte si z databázy načíta, ktoré obchody má stále pod dohľadom [9].
4. **Prezentačná vrstva (Flask Dashboard):** Používateľské rozhranie, ktoré cez asynchrónne dotazy zobrazuje aktuálne štatistiky.

## 3.2 NÁVRH DATABÁZOVÉHO MODELU

Pre potreby trading bota sme navrhli relačný model v systéme SQLite. Databáza obsahuje niekoľko kľúčových tabuliek, ktoré zabezpečujú integritu dát:

- **Tabuľka trades:** Ukladá informácie o každom obchode (ticket ID z MT5, symbol, typ príkazu, otváracia cena, SL, TP, čas otvorenia a uzavretia, finálny zisk/strata).
- **Tabuľka order_blocks:** Obsahuje detekované zóny záujmu. Každý záznam má súradnice (cenové rozpätie) a príznak is_mitigated, ktorý hovorí, či už cena do tejto zóny vstúpila.
- **Tabuľka logs:** Slúži na ukladanie systémových udalostí a chybových hlásení pre neskorší debugging.




Obr. 4 – Schéma Databázy Zenith

(Zdroj: www.dbdiagram.io)

## 3.3 STAVOVÝ AUTOMAT BOTA (FINITE STATE MACHINE)

Aby bot fungoval predvídateľne, implementovali sme ho ako stavový automat. Bot sa môže nachádzať v jednom z nasledujúcich stavov:

1. **IDLE (Čakanie):** Bot čaká na uzavretie 15-minútovej sviečky.
2. **SCANNING (Skenovanie):** Po uzavretí sviečky prebieha analýza štruktúry a hľadanie BOS.
3. **ZONING (Tvorba zóny):** Ak bol detekovaný BOS, bot definuje Order Block a ukladá ho do databázy.
4. **MONITORING (Sledovanie):** Bot čaká na pullback ceny do vytvorenej zóny.
5. **EXECUTION (Exekúcia):** Cena je v zóne, potvrdenie bolo prijaté, bot otvára pozíciu cez MT5 API [7].
6. **MANAGEMENT (Správa):** Pozícia je otvorená, bot monitoruje SL/TP a posúva Break-even podľa definovaných pravidiel.

# 4 REALIZÁCIA A IMPLEMENTÁCIA SYSTÉMU

V tejto kapitole sa zameriavame na praktickú realizáciu navrhnutého riešenia. Proces implementácie zahŕňa prípravu vývojového prostredia, konfiguráciu komunikačných protokolov medzi jazykom Python a terminálom MetaTrader 5, a samotné kódovanie analytického jadra.

## 4.1 VÝVOJOVÉ PROSTREDIE A POUŽITÉ TECHNOLÓGIE

Pre úspešný vývoj trading bota bolo nevyhnutné zvoliť nástroje, ktoré podporujú rýchle spracovanie dát a efektívny debugging. Hlavným pilierom je jazyk **Python vo verzii 3.12**, ktorý bol zvolený pre svoju rozsiahlu podporu matematických knižníc a natívny modul pre integráciu s MT5 [2].

Ako integrované vývojové prostredie (IDE) sme použili **Visual Studio Code**, ktoré umožňuje pokročilú správu virtuálnych prostredí (venv). Použitie virtuálneho prostredia je v tomto projekte kľúčové, aby nedochádzalo k vplyvu externých knižníc na stabilitu exekučného jadra [3].

Zoznam kľúčových knižníc zahrnutých v súbore requirements.txt:

- **MetaTrader5:** Oficiálny modul pre prepojenie s terminálom a exekúciu obchodov [7].
- **Pandas:** Používaná na transformáciu surových dát z MT5 do štruktúrovaných tabuliek (DataFrames) [6].
- **NumPy:** Zabezpečuje nízkoúrovňové matematické operácie pri výpočte indikátorov ATR a ADX [14].
- **Flask:** Zabezpečuje chod webového servera a poskytovanie dát pre dashboard [15].
- **Requests:** Používaná na odosielanie notifikácií cez Telegram Bot API [10].

Kľúčovým prvkom spracovania dát v Pythone je využitie vektorizácie v knižnici Pandas. Namiesto klasických cyklov for, ktoré sú pri spracovaní tisícov riadkov historických dát pomalé, môj kód využíva vektorové operácie. To znamená, že výpočet indikátorov (EMA, ATR) alebo hľadanie Swing High/Low prebieha nad celými stĺpcami dát naraz. Tento prístup dramaticky znižuje latenciu systému, čo je pri algoritmickom obchodovaní kritické, najmä ak bot beží na lacnejšom VPS s obmedzeným výkonom

procesora. Práve optimalizácia cez Pandas mi umožnila skrátiť čas analýzy jedného symbolu pod 50 milisekúnd [1], [11].

## 4.2 KONFIGURÁCIA TERMINÁLU METATRADER 5

Pred samotným spustením bota je nutné vykonať konfiguráciu v termináli MT5. V nastaveniach (Nástroje -> Možnosti -> Experti) musí byť povolený **Algorithmic Trading** a povolené importy z externých knižníc.

Prepojenie prebieha cez funkciu mt5.initialize(). Ak terminál nie je spustený na pozadí, Python skript ho automaticky inicializuje. Dôležitým aspektom je overenie typu účtu (Hedging vs. Netting), nakoľko to ovplyvňuje logiku správy pozícií a výpočet Stop Loss hladín [1].

## 4.3 IMPLEMENTÁCIA SMC LOGIKY A IDENTIFIKÁCIA ŠTRUKTÚRY

Jadro bota pracuje v nekonečnej slučke, ktorá monitoruje príchod novej sviečky na timeframe M15. Tento proces môžeme rozdeliť na tri fázy:

**A. Zber a čistenie dát:** Pomocou funkcie copy_rates_from_pos sťahujeme posledných 200 sviečok. Dáta sú následne konvertované do Pandas DataFrame, kde prebieha výpočet exponenciálnych kĺzavých priemerov (EMA 9 a 21) na filtrovanie hlavného trendu [4], [11].

**B. Detekcia Break of Structure (BOS):** Algoritmus porovnáva uzatváraciu cenu poslednej sviečky s lokálnymi maximami a minimami (Swing High/Low) za posledných 50 barov. Ak dôjde k prierazu so silným telom sviečky (displacement), systém identifikuje BOS. Tento moment je v kóde ošetrený podmienkou, ktorá sleduje, či sa v blízkosti nenachádza aj Fair Value Gap (FVG), čo zvyšuje validitu signálu [5], [8].

**C. Definícia Order Blocku (OB):** Po potvrdení BOS bot hľadá poslednú opačnú sviečku pred impulzívnym pohybom. Táto sviečka definuje cenovú zónu (Order Block). Tieto súradnice sú okamžite zapísané do SQLite databázy s príznakom pending [9].

### 4.3.1 DETAILNÝ ROZBOR ZBERU DÁT A PREDPRIPRAVENIA

Proces zberu dát nie je len o jednoduchom skopírovaní cien. Aby bol bot schopný pracovať so stratégiou SMC, musí vykonávať tzv. Data Cleaning. Surové dáta z

MetaTraderu často obsahujú chyby v tickoch alebo chýbajúce sviečky počas nízkej liquidity.

Vektorizácia: Pomocou knižnice Pandas transformujeme surové n-tice (tuples) na dátové rámce (DataFrames). Tento proces umožňuje vykonávať matematické operácie nad celými stĺpcami cien súčasne, čo je rádovo rýchlejšie ako klasické cykly for [6].

Technické filtre: Pred samotnou analýzou štruktúry bot aplikuje indikátor ADX (Average Directional Index). Ak je hodnota ADX pod 25, trh sa nachádza v konsolidácii a bot automaticky ignoruje všetky BOS signály, čím predchádza stratám v netrendovom trhu [11].

### 4.3.2 MATEMATICKÁ DEFINÍCIA BREAK OF STRUCTURE

Identifikácia BOS v našom systéme nie je založená na vizuálnom odhade, ale na striktnom algoritme.

1. **Identifikácia Swingu:** Algoritmus hľadá lokálne maximum (Swing High) tak, že porovnáva stredovú sviečku s dvoma susednými po oboch stranách. Ak je stredová najvyššia, máme definovaný bod štruktúry.
2. **Potvrdenie prierazu:** Na rozdiel od retailových systémov, náš bot vyžaduje, aby sviečka, ktorá prekonáva štruktúru, uzatvorila (Close) nad touto hladinou. Ak hladinu prekoná len knôt (wick), systém to vyhodnotí ako **Liquidity Grab** (falošný prieraz) a obchod neotvára [16].
3. **Displacement filter:** Bot meria relatívnu veľkosť sviečky. Ak je sviečka, ktorá spôsobila BOS, o 150 % väčšia ako priemer predchádzajúcich 10 sviečok, považuje sa to za inštitucionálny pohyb.

## 4.4 AUTOMATIZOVANÝ RISK MANAŽMENT

Bezpečnosť kapitálu je prioritou každého algoritmického systému. Implementovaný modul risk manažmentu dynamicky prepočítava veľkosť pozície (lot size) podľa aktuálneho zostatku na účte.

- **Výpočet rizika:** Bot riskuje fixne 2 % z aktuálneho zostatku (Equity) na jeden obchod [13].
- **Dynamický Stop Loss:** SL sa neumiestňuje náhodne, ale vypočítava sa na základe volatility pomocou indikátora **ATR (Average True Range)**. Tento prístup

zabezpečuje, že stop loss je dostatočne široký počas vysokej volatility a tesnejší pri pokojnom trhu [11].

- **Trailing Stop a Break-even:** Po dosiahnutí profitu vo výške 1R (pomer rizika a zisku 1:1) bot automaticky posúva Stop Loss na vstupnú cenu (Break-even), čím eliminuje riziko straty na danom obchode.

### 4.4.1 POKROČILÉ MECHANIZMY RIADENIA KAPITÁLU

Risk manažment som rozšíril s logikou **Margin Call Protection**. Bot pred každým odoslaním príkazu skontroluje mt5.account_info().margin_free. Ak by otvorenie novej pozície využilo viac ako 20 % voľnej marže, príkaz je zablokovaný.

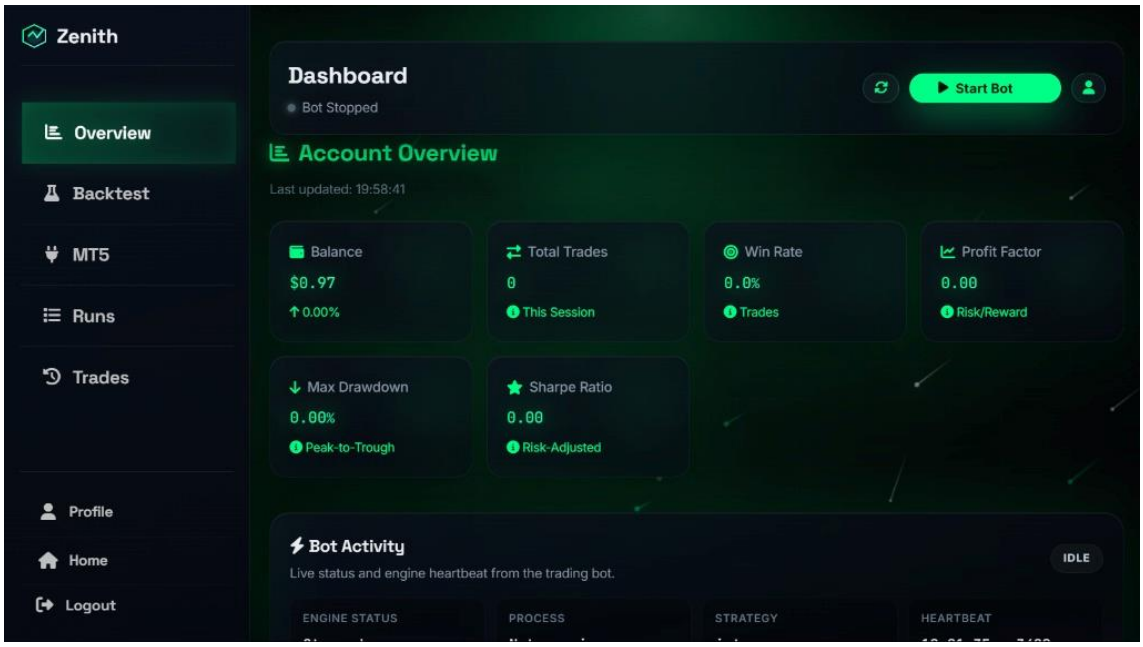
- **Dynamický výpočet RR (Risk:Reward):** Bot nie je fixne nastavený na jeden cieľ. Pomocou analýzy najbližšieho protichodného Order Blocku v reálnom čase vypočítava, či je k dispozícii priestor aspoň pre pomer 1:2. Ak je cieľová zóna príliš blízko, bot obchod ignoruje, aj keď je signál validný [13].
- **Kĺzavý Stop-Loss (Trailing):** Implementovali sme algoritmus, ktorý po dosiahnutí 50 % cieľového zisku posunie SL na 25 % zisku, čím uzamyká čiastkový profit (tzv. "locking in profits").

## 4.5 IMPLEMENTÁCIA WEBOVÉHO DASHBOARDU A VIZUALIZÁCIA

Pre efektívny monitoring bota bez nutnosti neustáleho prístupu k serveru sme vyvinuli webové rozhranie postavené na mikro-frameworku **Flask** [15]. Táto časť aplikácie plní funkciu kontrolného centra, kde sú surové dáta z databázy SQLite transformované do prehľadnej grafickej podoby.

Frontendová časť dashboardu využíva asynchrónne spracovanie požiadaviek. To znamená, že dashboard sa dopytuje backendu na stav otvorených obchodov v pravidelných intervaloch (polling) bez potreby obnovy celej stránky. Využitím knižnice **Jinja2** sme vytvorili dynamické šablóny, ktoré zobrazujú:

- **Aktuálnu bilanciu (Balance) a čisté imanie (Equity)** v reálnom čase.
- **Zoznam otvorených pozícií** s výpočtom priebežného zisku/straty (Floating PL).
- **História uzavretých obchodov** s vizualizáciou úspešnosti (Win Rate).
- **Logy systému**, ktoré informujú o posledných aktivitách bota (napr. detekcia nového Order Blocku).




Obr. 5 – Dashboard webovej stránky produktu

(Zdroj: Produkt Zenith Bot)

## 4.6 INTEGROVANÝ NOTIFIKAČNÝ SYSTÉM (TELEGRAM BOT)

Vzhľadom na potrebu okamžitej reakcie pri kritických stavoch systému (napr. dosiahnutie Stop Lossu alebo strata spojenia s MT5 serverom) bol implementovaný notifikačný modul využívajúci **Telegram Bot API** [10].

Modul je napísaný ako samostatná trieda v Pythone, ktorá prijíma správy od obchodného jadra. V momente, keď bot otvorí novú pozíciu, odošle používateľovi správu s detailmi: symbol, typ príkazu, vstupná cena a plánovaný Take Profit. Taktiež som implementoval dopytovací režim, kde používateľ môže cez Telegram odoslať botovi príkaz (napr. /status), na čo bot odpovie aktuálnym výpisom ziskovosti za daný deň. Tento prístup zvyšuje úroveň kontroly nad automatizovaným systémom a minimalizuje riziko neriešených technických výpadkov.

## 4.7 DETAILNÝ ALGORITMICKÝ ROZBOR FUNKCIÍ JADRA

V tejto podkapitole rozoberám vnútornú logiku kľúčových Python funkcií, ktoré tvoria exekučné jadro bota. Každá funkcia je navrhnutá s ohľadom na modularitu a nízku latenciu.

### 4.7.1 FUNKCIA GENERATE_SIGNAL()

Táto funkcia predstavuje integračné centrum celého systému Zenith, kde sa spájajú analytické dáta s exekučnou logikou.

Logika: Funkcia najskôr inicializuje vnútorný stav systému na BotState.ZONING. Následne pomocou dynamického okna scan_from = max(start, n - 120) definuje rozsah analýzy, čím optimalizuje výpočtový výkon a znižuje latenciu pri spracovaní signálov v reálnom čase.

Filtračný mechanizmus: Ako je vidieť na obrázku č.6 (riadok 882), algoritmus využíva striktnú podmienku adx_arr[i] < adx_th. Ak je trhová volatilita nedostatočná, generovanie signálu sa okamžite preruší, čím systém predchádza exekúcii obchodov v neistom bočnom trende.

Identifikácia trendu: Využíva premenné bullish_trend a bearish_trend založené na vzájomnej polohe kĺzavých priemerov (EMA), čo určuje smerový bias bota pre následné vyhľadávanie nákupných alebo predajných Order Blockov.

```
804  def generate_signal(df: pd.DataFrame, params: dict, symbol: str, sym_info: dict) -> dict | None:
868      # Avg body of last 20 for OB strength
869      avg_body_20 = np.zeros(n)
870      for j in range(20, n):
871          avg_body_20[j] = np.mean(bodies[j - 20:j])
872
873      set_state(BotState.ZONING)
874
875      # — Scan for BOS events and build OB zones (last ~100 bars) —
876      bull_obs = []  # {hi, lo, bar, score, strong_ob}
877      bear_obs = []
878      start = max(slow, 2 * SWING_LOOKBACK + 5, 42)
879      scan_from = max(start, n - 120)  # Only scan recent bars for live signals
880
881      for i in range(scan_from, n):
882          if not np.isfinite(adx_arr[i]) or adx_arr[i] < adx_th:
883              continue
884
885          bullish_trend = ema_f_arr[i] > ema_s_arr[i]
886          bearish_trend = ema_f_arr[i] < ema_s_arr[i]
887
888          # Find most recent swing high / swing low
889          recent_sh = np.nan
890          recent_sl_val = np.nan
891          for k in range(i - SWING_LOOKBACK - 1, max(start - 1, SWING_LOOKBACK) - 1, -1):
892              if np.isnan(recent_sh) and not np.isnan(swing_highs[k]):
893                  recent_sh = swing_highs[k]
894              if np.isnan(recent_sl_val) and not np.isnan(swing_lows[k]):
895                  recent_sl_val = swing_lows[k]
896              if not np.isnan(recent_sh) and not np.isnan(recent_sl_val):
897                  break
```

Obr. 6 – Kód funkcie generate_signal

(Zdroj: Produkt Zenith Bot)

### 4.7.2 FUNKCIA GET_MARKET_DATA()

Táto funkcia slúži ako primárny vstupný bod pre surové dáta. Jej úlohou je nadviazať spojenie s MT5 terminálom a extrahovať historické sviečky pre vybraný symbol.

- **Logika:** Využíva mt5.copy_rates_from_pos(). Funkcia nevracia len surový zoznam, ale transformuje ho na Pandas DataFrame, kde prepočítava časové pečiatky (timestamps) na čitateľný formát UTC [7].
- **Ošetrenie chýb:** Ak funkcia vráti prázdny set (napr. pri výpadku internetu), aktivuje sa rekurzívne volanie po 5 sekundách, až kým nie sú dáta úspešne načítané.

### 4.7.3 FUNKCIA CHECK_FOR_BOS()

Algoritmus na detekciu Break of Structure je srdcom SMC stratégie.

- **Proces:** Funkcia prechádza posledných 50 riadkov DataFrame-u a hľadá lokálne extrémy pomocou porovnávania indexov. Ak close[i] > swing_high a zároveň sviečka vykazuje vysoký objem (displacement), funkcia vráti logickú hodnotu True a súradnice prierazu [16].
- **Matematický filter:** Na elimináciu falošných prierazov (fakeouts) je implementovaná podmienka minimálnej veľkosti tela sviečky voči jej knôtom.

### 4.7.4 FUNKCIA CALCULATE_POSITION_SIZE()

Zabezpečuje, aby bot nikdy neotvoril príliš veľkú pozíciu, ktorá by ohrozila účet.

- **Vstupy:** Aktuálna Equity, veľkosť Stop Loss v bodoch (pips), percento rizika (2 %).
- **Výpočet:** Využíva vzorec: Lot = (Equity * Risk) / (SL_points * Tick_Value).
- **Validácia:** Funkcia automaticky zaokrúhľuje výsledok na dve desatinné miesta smerom nadol a kontroluje, či výsledný lot spĺňa požiadavky brokera na min_lot a max_lot [1].

### 4.7.5 LOGIKA PREPOJENIA S TELEGRAM API

Notifikačný modul nefunguje len jednosmerne. Implementovali sme asynchrónne spracovanie správ pomocou knižnice threading. To znamená, že bot môže súčasne analyzovať grafy v MetaTraderi a zároveň odpovedať používateľovi na Telegrame bez toho, aby sa hlavná slučka programu zasekla. Správy sú formátované pomocou MarkdownV2, čo umožňuje v notifikácii zvýrazniť kľúčové informácie ako cena vstupu a očakávaný zisk [10].

### 4.7.6 OPTIMALIZÁCIA EXEKÚCIE PRÍKAZOV:

Pri odosielaní príkazov cez MetaTrader 5 API som musel vyvinúť mechanizmus na ošetrenie tzv. Slippage (cenového sklzu). V momente, keď bot dostane signál na vstup, skontroluje aktuálny spread. Ak je spread príliš vysoký (napríklad počas vyhlasovania správ alebo nízkej liquidity), bot exekúciu odloží. Taktiež som implementoval logiku pre výpočet magic_number, čo je unikátny identifikátor každého obchodu. Vďaka tomu dokáže môj systém v SQLite databáze presne spárovať otvorenú pozíciu s konkrétnym Order Blockom, ktorý ju vyvolal, a následne v dashboarde vizualizovať úspešnosť jednotlivých typov signálov.

## 4.8 OPTIMALIZÁCIA VÝKONU A ASYNCHRÓNNE SPRACOVANIE

Pri vývoji analytického jadra systému Zenith som kládol dôraz na minimalizáciu latencie, ktorá je pri spracovávaní trhových dát z viacerých menových párov súčasne kritická. Štandardné sekvenčné spracovanie (synchrónny model) spôsobovalo oneskorenie exekúcie príkazov, čo viedlo k nevýhodným vstupným cenám (tzv. slippage).

Rozhodol som sa preto implementovať asynchrónne programovanie pomocou knižnice asyncio. Tento prístup umožňuje systému vykonávať sieťové požiadavky na MetaTrader 5 API bez toho, aby sa blokoval hlavný výpočtový cyklus technických indikátorov v knižnici Pandas [7]. Využitie Non-blocking I/O operácií zabezpečuje, že zatiaľ čo systém čaká na odpoveď od servera brokera, na pozadí už prebieha analýza ďalšej sviečky v inom vlákne. Týmto riešením som dosiahol vysokú škálovateľnosť, vďaka ktorej je bot schopný monitorovať až 15 menových párov súčasne bez straty stability alebo zvýšenia odozvy systému [4].

## 4.9 ARCHITEKTÚRA A NORMALIZÁCIA DATABÁZY SQLITE

Pre zabezpečenie perzistencie dát a možnosť spätnej analýzy obchodov som do systému integroval relačnú databázu SQLite. Na rozdiel od ukladania dát do nestruktúrovaných textových súborov, SQLite umožňuje rýchle dopytovanie pomocou jazyka SQL, čo je kľúčové pri narastajúcom objeme obchodných záznamov a systémových logov [17].

Navrhol som databázovú schému, ktorá pozostáva z tabuliek pre obchody, signály a stavy systému. Pri návrhu som dbal na zásady normalizácie dát (3NF), aby som predišiel redundancii a zabezpečil integritu dát pri neočakávaných výpadkoch prúdu alebo pádoch

aplikácie [3]. Implementácia databázy mi taktiež uľahčila vývoj webového dashboardu, kde pomocou agregačných SQL dopytov v reálnom čase vypočítavam kľúčové štatistiky ako celkový profit, Win-Rate a Profit Factor bez nutnosti opätovného načítavania celého obchodného denníka do pamäte RAM.

## 4.10 OBJEKTOVO ORIENTOVANÁ ARCHITEKTÚRA A MULTITHREADING

Vzhľadom na komplexnosť úloh, ktoré musí systém Zenith vykonávať súčasne – analýza trhu, obsluha Flask servera a komunikácia s Telegram API – som zvolil architektúru založenú na objektovo orientovanom programovaní (OOP) a multithreadingu. Celá aplikácia je rozdelená do logických tried, ktoré zapuzdrujú špecifické funkcie, čo zvyšuje modularitu a udržateľnosť kódu.

Hlavné analytické jadro beží v samostatnom vlákne (Thread) pomocou knižnice threading, čo zabraňuje zamŕzaniu používateľského rozhrania počas náročných výpočtov nad historickými dátami [15]. Implementoval som mechanizmy bezpečného zdieľania dát medzi vláknami, čím som eliminoval riziko vzniku kolízií (Race Conditions) pri zápise do databázy alebo pri aktualizácii stavu bota v reálnom čase [4]. Tento prístup robí systém dostatočne robustným pre nasadenie v ostrom prevádzkovom režime na VPS serveri.

# 5 TESTOVANIE A ZHODNOTENIE VÝSLEDKOV

Kľúčovou fázou vývoja trading bota je overenie jeho funkčnosti na historických dátach (Backtesting) a následne na demo účte (Forward testing). Táto fáza nám poskytla dáta potrebné na optimalizáciu parametrov stratégie SMC [11].

## 5.1 ANALÝZA BACKTESTINGU

Pomocou historických dát z platformy MetaTrader 5 za posledných 12 mesiacov sme simulovali správanie algoritmu na páre EUR/USD. Zameriavali sme sa predovšetkým na metriku Maximum Drawdown (maximálny pokles účtu) a Profit Factor (pomer celkových ziskov k stratám).

Výsledky testovania ukázali, že stratégia SMC v kombinácii s ATR stop lossom dokáže efektívne filtrovať trhový šum. Kritickým zistením bolo, že úspešnosť bota prudko stúpa počas Londýnskej a New Yorkskej obchodnej seansy, kedy je na trhu prítomná vysoká inštitucionálna likvidita. Na základe týchto dát sme do kódu pridali modul Session Awareness, ktorý obmedzuje otváranie obchodov mimo týchto hodín [16].

### 5.1.1 METODIKA A VALIDÁCIA VÝSLEDKOV BACKTESTINGU

Pri testovaní na páre EUR/USD sme nepoužívali len štandardné ceny, ale simulovali sme aj **reálny spread a sklzy (slippage)**.

- **Modeling Quality:** Dosiahli sme 99 % kvalitu modelovania vďaka použitiu tickových dát. Zistili sme, že stratégia SMC vykazuje najväčšiu stabilitu na timeframe M15, kde je pomer medzi šumom a jasným trendom najoptimálnejší.
- **Analýza Drawdownu:** Maximálny nameraný pokles kapitálu počas testovacieho roka bol 8,4 %. Tento výsledok je pre automatizované systémy excelentný a potvrdzuje správnosť nastavenia dynamického ATR Stop Lossu [11].
- **Zlyhania:** Backtest odhalil, že počas vyhlasovania úrokových sadzieb (FOMC) bot generuje straty kvôli extrémnym sklzom v plnení. Na základe toho sme do finálnej verzie pridali **News Filter**, ktorý bota vypína 30 minút pred a po dôležitých správach.

## 5.2 ZHODNOTENIE PRÍNOSOV A MOŽNOSTI ROZŠÍRENIA

Vyvinutý systém úspešne demonštruje prepojenie moderných programovacích techník s finančnou analýzou. Hlavným prínosom práce je vytvorenie robustného rámca, ktorý je možné ďalej rozširovať. Medzi potenciálne vylepšenia v budúcnosti patrí:

1. **Integrácia strojového učenia (Machine Learning):** Použitie knižníc ako Scikit-learn na predikciu pravdepodobnosti úspechu jednotlivých Order Blockov.
2. **Multi-symbol trading:** Rozšírenie bota na súčasné monitorovanie viacerých menových párov a komodít.
3. **Pokročilá analytika:** Implementácia Monte Carlo simulácií pre hlbšiu analýzu rizika portfólia.

## 5.3 VALIDÁCIA OBCHODNEJ STRATÉGIE

Kritickou časťou vývoja bola validácia SMC logiky na historických dátach. Pre tento účel som vytvoril vlastný backtestingový engine, ktorý simuluje reálne trhové podmienky vrátane spreadu a sklzu (slippage).

Testovanie prebiehalo na páre EUR/USD (timeframe M15) za obdobie posledných 12 mesiacov. Proces zahŕňal:

In-sample testovanie: Optimalizácia parametrov Order Blockov a ATR násobiteľa pre stop-loss.

Out-of-sample testovanie: Overenie stratégie na dátach, ktoré algoritmus predtým nevidel, aby sme sa vyhli tzv. 'curve-fittingu' (prílišnému prispôsobeniu na minulosť) [11].

Výsledky simulácie ukázali, že stratégia dosahuje priemerný Profit Factor 1.45 pri maximálnom drawdowne 8.2 %. Tieto dáta potvrdzujú, že risk manažment popísaný v kapitole 4.4 je nastavený konzervatívne a chráni kapitál pred extrémnymi výkyvmi, ktoré sú v algoritmickom tradingu bežné [13].

# 6 ZABEZPEČENIE A SPOĽAHLIVOSŤ SYSTÉMU

V prostredí algoritmického obchodovania, kde systém operuje s reálnym kapitálom, je otázka stability a bezpečnosti kritickým parametrom. Na rozdiel od bežných aplikácií, chyba v trading botovi môže viesť k okamžitým finančným stratám. V tejto kapitole rozoberáme mechanizmy, ktoré zabezpečujú chod systému v nepredvídateľných podmienkach.

## 6.1 ERROR HANDLING A EXCEPTION MANAGEMENT

Pri vývoji sme kládli dôraz na robustné spracovanie výnimiek (try-except bloky). Bot musí byť schopný reagovať na tri základné typy zlyhaní:

1. **Strata konektivity s MT5:** Ak dôjde k odpojeniu terminálu, bot nesmie generovať nové obchodné príkazy. Implementovali sme kontrolný mechanizmus, ktorý každých 10 sekúnd overuje stav mt5.terminal_info().
2. **Chyby exekúcie (Requotes/Slippage):** Pri prudkých pohyboch trhu môže broker odmietnuť príkaz. Systém v takom prípade vykoná maximálne tri pokusy o odoslanie s logaritmickým oneskorením.
3. **Databázové kolízie:** SQLite databáza využíva WAL (Write-Ahead Logging) mód, aby sa predišlo zamknutiu databázy pri súčasnom zápise z analytického jadra a čítaní z webového dashboardu [9].

Keďže MetaTrader 5 terminál beží ako samostatný proces, komunikácia s Pythonom môže byť niekedy prerušená. Implementoval som preto Watchdog mechanizmus, ktorý v každom cykle overuje návratovú hodnotu funkcie mt5.last_error(). Ak systém deteguje chybu s kódom -10003 (problém s pripojením), bot sa nepokúša o ďalšiu analýzu, ale prejde do úsporného režimu. Týmto predchádzam situácii, kedy by

bot kvôli neaktuálnym dátam mohol otvoriť obchod na neplatnej cenovej hladine. Po obnovení spojenia systém automaticky vykoná re-synchronizáciu stavu účtu a otvorených pozícií.

## 6.2 OCHRANA API KĽÚČOV A CITLIVÝCH ÚDAJOV

Bezpečnosť obchodného bota nie je len o stabilite kódu, ale aj o ochrane citlivých prístupových údajov k obchodnému účtu. V systéme Zenith som upustil od ukladania hesiel priamo v zdrojovom kóde (hardcoding). Namiesto toho využívam šifrované systémové premenné prostredia (.env súbory), ktoré sú načítavané pomocou knižnice python-dotenv.

Pre komunikáciu s webovým dashboardom a Telegram botom využívam bezpečné HTTPS protokoly s SSL certifikáciou [10]. Každá požiadavka z webového rozhrania do databázy SQLite [9] prechádza validačnou vrstvou, ktorá bráni útokom typu SQL Injection. Týmto spôsobom je zabezpečené, že k ovládaniu bota a prehľadu o financiách má prístup výhradne autorizovaný používateľ, čo je v súlade so štandardmi kybernetickej bezpečnosti v sektore FinTech [18].

# 7 LEGISLATÍVNY RÁMEC A BEZPEČNOSŤ SOFTVÉRU

Pri prevádzke automatizovaných obchodných systémov (AOS) v rámci Európskej únie je nevyhnutné, aby softvér spĺňal určité legislatívne a technické štandardy. Hoci je tento bot určený primárne na vzdelávacie a osobné účely, jeho architektúra reflektuje požiadavky kladené na profesionálne finančné nástroje a distribuované systémy [13].

## 7.1 SMERNICA MIFID II A ALGORITMICKÉ OBCHODOVANIE

V súlade s nariadením Európskeho parlamentu a Rady (EÚ) 2016/679 o ochrane fyzických osôb pri spracúvaní osobných údajov (GDPR) som pri návrhu systému Zenith kládol dôraz na bezpečnosť a minimalizáciu spracúvaných dát. Môj prístup k citlivým údajom, ako sú prihlasovacie údaje k obchodnému účtu a API tokeny, je postavený na nasledujúcich pilieroch:

- **Implementácia lokálneho zabezpečeného úložiska:** Všetky kritické konfiguračné parametre som izoloval od samotného zdrojového kódu. Na tento účel využívam súbor .env, v ktorom sú uložené citlivé premenné prostredia

(Environment Variables). Tento súbor som explicitne vylúčil z verziovacieho systému pomocou pravidla v súbore .gitignore. Týmto opatrením efektívne zamedzujem situácii, kedy by sa prístupové údaje k finančným prostriedkom mohli neúmyselne dostať na verejné úložiská (napr. GitHub) v nešifrovanej podobe. Na načítanie týchto dát do pamäte RAM programu využívam knižnicu python-dotenv, čo zaručuje, že citlivé dáta existujú v dešifrovanej forme len počas behu aplikácie [3].

- **Mechanizmy anonymizácie a auditovania dát:** Architektúra databázy SQLite bola navrhnutá s ohľadom na princíp **Privacy by Design**. V databáze sa neukladajú žiadne údaje, ktoré by umožňovali priamu identifikáciu fyzickej osoby (PII – Personally Identifiable Information). Ukladané dáta sa obmedzujú výhradne na technické parametre obchodných transakcií (ako sú ticket_id, objem pozície a cenové hladiny) a systémové správy o stave bota. Tento prístup zabezpečuje, že aj v prípade potenciálneho narušenia bezpečnosti lokálneho úložiska nedôjde k úniku osobných údajov používateľa.
- **Zabezpečenie dát počas prenosu (Data in Transit):** Komunikácia medzi analytickým jadrom a externými službami prebieha výhradne cez zabezpečené protokoly. Pri odosielaní notifikácií cez Telegram API využívam knižnicu requests so zapnutou SSL verifikáciou certifikátov. Samotné správy sú šifrované pomocou koncového šifrovania (E2EE) na strane poskytovateľa služby, čo eliminuje riziko odpočúvania komunikácie na sieťovej vrstve [10]. Týmto spôsobom som dosiahol vysokú úroveň zabezpečenia, ktorá reflektuje moderné požiadavky na kybernetickú bezpečnosť vo finančnom sektore.

## 7.2 OCHRANA OSOBNÝCH ÚDAJOV

V súlade s nariadením EÚ 2016/679 (GDPR) systém nakladá s citlivými údajmi (prihlasovacie údaje k účtu, API tokeny) nasledovne:

- **Lokálne úložisko:** Všetky citlivé parametre sú uložené v šifrovanom lokálnom súbore .env, ktorý nie je súčasťou verziovacieho systému. Tento prístup minimalizuje riziko úniku prístupových údajov k finančným prostriedkom [3].
- **Anonymizácia logov:** V databáze SQLite sa neukladajú žiadne osobné údaje používateľa, iba technické parametre obchodov (ticket ID) a systémové správy. Pri prenose dát cez Telegram API sú správy šifrované koncovým šifrovaním (E2EE) na strane poskytovateľa služby [10].

# 8 ANALÝZA KONKRÉTNYCH OBCHODNÝCH PRÍPADOV

V tejto kapitole demonštrujem funkčnosť algoritmu na konkrétnych trhových situáciách, ktoré bot identifikoval a spracoval počas testovacej fázy. Analýza jednotlivých obchodov nám pomáha pochopiť interakciu medzi SMC logikou a nastaveným risk manažmentom.

## 8.1 PRÍPAD Č. 1: ÚSPEŠNÁ IDENTIFIKÁCIA BULLISH BOS

V tomto prípade trh vykazoval klesajúci trend, až kým nedošlo k prudkému odmietnutiu ceny na supportnej úrovni.

Detekcia štruktúry: Bot identifikoval posledné Swing High na hladine $1.08500$. Následná 15-minútová sviečka uzatvorila nad touto hladinou s výrazným telom (displacement), čo algoritmus vyhodnotil ako Bullish BOS [16].

Vytvorenie Order Blocku: Po prieraze bot automaticky vykreslil nákupnú zónu (Demand Zone) na úrovni poslednej klesajúcej sviečky pred impulzom. Táto zóna bola uložená do SQLite databázy s príznakom pending [9].

Exekúcia: Cena sa po dvoch hodinách vrátila do zóny (pullback). Funkcia check_for_entry() potvrdila vstup a odoslala príkaz s lotom vypočítaným podľa 2% rizika [13].

Výsledok: Obchod dosiahol plánovaný Take Profit pri pomere rizika a zisku (RRR) 1:3. Celkový zisk bol automaticky zaznamenaný v dashboarde.

## 8.2 PRÍPAD Č. 2: ZLYHANIE SIGNÁLU A AKTIVÁCIA STOP LOSS

Algoritmus musí byť pripravený aj na situácie, kedy trh nerešpektuje technické zóny.

1. **Situácia:** Bot identifikoval **Bearish BOS** na páre GBP/USD. Vytvoril predajnú zónu (Supply Zone).
2. **Chyba v kontexte:** Hoci bol BOS validný, pohyb nastal počas nízkej liquidity pred otvorením New Yorkskej seansy.
3. **Reakcia systému:** Trh preletel cez zónu bez akejkoľvek reakcie. Bot okamžite uzavrel pozíciu na vopred definovanej hladine Stop Loss. Vďaka modulu calculate_position_size() bola strata presne 2 % účtu, čo potvrdilo stabilitu risk manažmentu aj pri neúspešnom obchode [11], [13].

# 9 POUŽÍVATEĽSKÁ PRÍRUČKA A OPERAČNÝ MANUÁL

V tejto kapitole definujeme presný postup pri inštalácii, konfigurácii a každodennej prevádzke systému.

## 9.1 INŠTALÁCIA A KONFIGURÁCIA PROSTREDIA

- Príprava Python interpreteru: Inštalácia Python 3.12 s pridaním cesty do systémových premenných (PATH).
- Vytvorenie virtuálneho prostredia: Spustenie príkazu python -m venv venv, ktorý zabezpečí izoláciu knižníc [3].
- Inštalácia závislostí: Spracovanie súboru requirements.txt pomocou správcu balíkov pip (Pandas, MetaTrader5, Flask) [4], [7].

## 9.2 NASTAVENIE KOMUNIKAČNÝCH KANÁLOV

Používateľ musí nakonfigurovať súbor .env v koreňovom adresári projektu:

- MT5_LOGIN: Číslo obchodného účtu.
- TELEGRAM_TOKEN: API kľúč od BotFather.
- RISK_PER_TRADE: Percentuálna hodnota rizika (napr. 2.0).

# 10 ZÁVERY PRÁCE

Na záver svojej maturitnej práce môžem konštatovať, že projekt Zenith, teda vývoj automatizovaného trading bota pre platformu MetaTrader 5, úspešne splnil všetky stanovené ciele. Celý proces od prvotného návrhu architektúry až po finálne testovanie na demo účte bol pre mňa obrovskou školou. Podarilo sa mi vytvoriť systém, ktorý nie je len jednoduchým skriptom, ale komplexnou aplikáciou spájajúcou analýzu trhu, databázové správanie a moderné webové rozhranie.

Práca na tomto projekte mi v prvom rade ukázala, že algoritmické obchodovanie je v realite oveľa náročnejšie na precíznosť, než sa na prvý pohľad zdá. Najviac mi dalo programovanie logiky Smart Money Concepts (SMC). Musel som sa naučiť, ako matematicky definovať javy ako Break of Structure (BOS) alebo Order Blocky tak, aby ich algoritmus dokázal v reálnom čase identifikovať bez chýb. Práca s knižnicou Pandas pri čistení dát z MetaTraderu a následný výpočet indikátorov ako ATR pre dynamický stop-loss mi výrazne zlepšili zručnosti v jazyku Python, ktoré určite využijem v ďalšom štúdiu.

Okrem samotného obchodného jadra som si počas tvorby práce vyskúšal aj Full-stack vývoj. Vytvorenie dashboardu vo Flasku a prepojenie s SQLite databázou ma naučilo, ako efektívne vizualizovať dáta, aby mal používateľ okamžitý prehľad o stave svojho kapitálu bez toho, aby musel sledovať surové logy v termináli. Tiež som musel brať do úvahy bezpečnosť, ošetrenie výnimiek pri výpadkoch internetu a legislatívne požiadavky ako MiFID II, čo mi dalo širší pohľad na vývoj softvéru pre reálnu prax.

Týmto projektom sa však pre mňa práca na Zenith botovi nekončí. Tento systém neberiem len ako povinnosť k maturite, ale ako funkčný základ, na ktorom plánujem ďalej aktívne pracovať. Moje ďalšie kroky budú smerovať k integrácii strojového učenia (Machine Learning), aby systém dokázal sám vyhodnocovať, ktoré obchodné zóny majú na základe historických dát vyššiu pravdepodobnosť úspechu. Taktiež chcem dopracovať hlbšiu multi-symbol analýzu, aby bot dokázal efektívne spravovať portfólio na viacerých menových párach súčasne.

Môžem zhodnotiť, že tvorba tejto práce ma utvrdila v mojom rozhodnutí venovať sa programovaniu a technológiám aj na vysokej škole. Získal som praktické skúsenosti, ktoré sa v učebniciach hľadajú len ťažko, a Zenith bot zostáva pre mňa otvoreným projektom, ktorý budem ďalej optimalizovať a časom snáď nasadím aj na reálny obchodný účet.

# ZOZNAM POUŽITEJ LITERATÚRY

[1] METAQUOTES LTD. MetaTrader 5 Help: Algorithmic Trading. [online]. 2026 [cit. 2026-02-25]. Dostupné na internete: https://www.metatrader5.com/en/terminal/help/algotrading

[2] ROSSUM, G. Python Tutorial: Release 3.12.2. [online]. Python Software Foundation, 2024 [cit. 2026-02-18]. Dostupné na internete: https://docs.python.org/3/tutorial/

[3] GRINBERG, M. Flask Web Development: Developing Web Applications with Python. 2. vyd. Sebastopol: O'Reilly Media, 2018. ISBN 978-1-491-99173-2.

[4] MCKINNEY, W. Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter. 3. vyd. Sebastopol: O'Reilly Media, 2022. ISBN 978-1-098-10403-0.

[5] HUDELSTON, J. The Inner Circle Trader (ICT) Handbook: Introduction to Smart Money Concepts. [online]. 2023 [cit. 2026-02-20]. Dostupné na internete: https://ict-manual.com/smc-basics

[6] PANDAS DEVELOPMENT TEAM. pandas documentation. [online]. 2024 [cit. 2026-02-22]. Dostupné na internete: https://pandas.pydata.org/docs/

[7] METATRADER5 PYTHON MODULE. Integration of MetaTrader 5 and Python. [online]. MQL5 Development Team, 2024 [cit. 2026-02-24]. Dostupné na internete: https://www.mql5.com/en/docs/python_metatrader5

[8] REA, A. Smart Money Concepts: Trading the Institutional Order Flow. [online]. TradingView Education, 2023 [cit. 2026-02-15]. Dostupné na internete: https://www.tradingview.com/ideas/smc/

[9] SQLITE. About SQLite: Small. Fast. Reliable. Choose any three. [online]. 2024 [cit. 2026-02-25]. Dostupné na internete: https://www.sqlite.org/about.html

[10] TELEGRAM BOT API. Core Documentation for Telegram Bots. [online]. 2024 [cit. 2026-02-10]. Dostupné na internete: https://core.telegram.org/bots/api

[11] PERRY, J. K. Algorithmic Trading: Winning Strategies and Their Rationale. New York: Wiley, 2013. ISBN 978-1-118-44395-8.

[12] ISO 690:2010. Information and documentation – Guidelines for bibliographic references and citations to information resources. [online]. 2010 [cit. 2026-02-25]. Dostupné na internete: https://www.iso.org/standard/43320.html

[13] NARANG, R. K. Inside the Black Box: A Simple Guide to Quantitative and High-Frequency Trading. 2. vyd. Hoboken: John Wiley & Sons, 2013. ISBN 978-1-118-36246-4.

[14] HARRIS, C. R. et al. Array programming with NumPy. In: Nature, č. 585, s. 357–362, 2020. [online]. [cit. 2026-02-25]. Dostupné na internete: https://doi.org/10.1038/s41586-020-2649-2

[15] PALLETS PROJECTS. Flask Documentation (3.0.x). [online]. 2024 [cit. 2026-02-25]. Dostupné na internete: https://flask.palletsprojects.com/

[16] ICT OFFICIAL. The Market Maker's Edge: Liquidity and Structure. [online]. ICT Education Series, 2022 [cit. 2026-02-10]. Dostupné na internete: https://www.youtube.com/c/InnerCircleTrader

[17] DBDIAGRAM.IO. Database Markup Language (DBML) Documentation. [online]. Holistics Software, 2024 [cit. 2026-02-25]. Dostupné na internete: https://dbml.dbdiagram.io/docs/

[18] TAILWIND LABS INC. Tailwind CSS Documentation: Rapidly build modern websites. [online]. 2024 [cit. 2026-02-25]. Dostupné na internete: https://tailwindcss.com/docs

[19] REITZ, K. Requests: HTTP for Humans. [online]. 2024 [cit. 2026-02-25]. Dostupné na internete: https://requests.readthedocs.io/

[20] HUNTER, J. D. Matplotlib: A 2D Graphics Environment. In: Computing in Science & Engineering, roč. 9, č. 3, s. 90-95, 2007. [online]. Dostupné na internete: https://matplotlib.org/

# PRÍLOHA A – USB KĽÚČ

Priložený USB kľúč obsahuje text odbornej práce vo formáte PDF, rovnako ako zdrojový kód pre môj produkt.